
Compte Rendu de Réunion

Projet Filé

Réunion hebdomadaire de suivi de projet

	Date	Vendredi 13 février 2026
	Horaire	14 h 30 — 15 h 30 (GMT+01 :00)
	Lieu	Réunion Microsoft Teams
	Organisateur	Emilien Mottet <code>emilien.mottet@michelin.com</code>
	Récurrence	Tous les vendredis, du 06/02/2026 au 29/05/2026

Participants

Encadrant	Emilien Mottet	<code>emilien.mottet@michelin.com</code>
Élève	Widad Benkhadra	<code>widad.benkhadra@grenoble-inp.org</code>
Élève	Ayoub Touati	<code>ayoub.touati@grenoble-inp.org</code>
Élève	Mohammed Lrhorfi	<code>mohammed.lrhorfi@grenoble-inp.org</code>

Table des matières

1	Déroulement de la réunion	3
2	Démonstrations techniques	3
2.1	Démo Python — HTTP / Ollama	3
2.2	Démo Rust — Framework RIG	3
2.3	Commentaires de l'encadrant	3
3	Outils et technologies discutés	4
4	Attentes pour la semaine prochaine	4
5	Actions envisagées	5
6	Processus de développement et collaboration	5
6.1	Intégration Continue / Déploiement Continu (CI/CD)	5
6.2	Polyvalence de l'équipe	5
7	Rappel de la feuille de route du projet	6

1. Déroulement de la réunion

La réunion s’est déroulée selon l’ordre du jour suivant :

Horaire	Durée	Sujet
14 h 35 — 14 h 40	5 min	Présentation du planning prévisionnel
14 h 40 — 14 h 55	15 min	Démonstrations techniques (Ollama, scripts Python & Rust)
14 h 55 — 15 h 10	15 min	Attentes pour la semaine prochaine & conseils de l’encadrant

TABLE 1 – Ordre du jour de la réunion

2. Démonstrations techniques

2.1. Démo Python — HTTP / Ollama

Présentation d’un script Python se connectant via HTTP à **Ollama**. Ce script envoie un prompt au modèle **LLaMA 3.2** (1 milliard de paramètres) et reçoit une réponse.

 **Temps de réponse mesuré : 24 secondes.**

Un petit template de test pour les tickets IT a également été présenté.

2.2. Démo Rust — Framework RIG

Démonstration (inachevée) du même script implémenté en **Rust** avec le framework **RIG**.

 **Temps d’exécution mesuré : 38 secondes** — performance en deçà des attentes.
Une révision urgente de cette implémentation est nécessaire.

2.3. Commentaires de l’encadrant

L’encadrant a apporté les précisions suivantes :

- La fonction `tester_support_it` relève de l’**observabilité** : comprendre le comportement du système en temps réel.
- Protocole recommandé : **OpenTelemetry** — <https://opentelemetry.io/>
- Ressource complémentaire sur l’observabilité des agents IA : <https://opentelemetry.io/blog/2025/ai-agent-observability/>

— Explorer **LLaMA.cpp** (<https://github.com/ggml-org/llama.cpp>), implémentation C++ offrant de meilleures performances.

3. Outils et technologies discutés

Domaine	Description
Observabilité	Intégration d'OpenTelemetry pour le monitoring du système.
Modèles LLM	Exploration de LLaMA.cpp (inférence C++ haute performance).
Tests & Qualité	Ajout de llm-inspector pour la validation des tickets IT.
Architecture Réseau	Utilisation de GitHub Copilot en tant que <i>reverse proxy</i> (proxy devant Ollama).
Ressources à explorer	GitHub Copilot Pro, QwenCoder, Minimax, GPT-5, GitHub Models.

TABLE 2 – Technologies et outils abordés

4. Attentes pour la semaine prochaine

L'équipe doit sélectionner **3 points parmi les 5** suivants à traiter pour la prochaine réunion :

- Point 1 Exploration de LLM supplémentaires** — Identifier de nouveaux modèles (Minimax, GPT-5, GitHub Models) et comparer leurs performances.
- Point 2 Construction de l'agent** — Début de l'implémentation de l'architecture agentique du système.
- Point 3 CI/CD & Tests** — Mettre en place un pipeline d'intégration continue ; commencer par des tests simples (ex. : vérification d'un mot-clé dans la réponse).
- Point 4 OpenTelemetry** — Étudier le protocole et planifier son intégration pour l'observabilité.
- Point 5 Déploiement & LLM Inspector** — Trouver un serveur de déploiement ; intégrer LLM Inspector pour tester les biais (benchmark avec *system prompt*, agent et LLM).

5. Actions envisagées

✓ Récapitulatif des actions à mener

#	Action	Responsable	Priorité
1	Implémenter des tests <i>badges</i> (statuts de tests) à chaque push sur GitHub.	Équipe	Haute
2	Réviser l'implémentation Rust/RIG pour améliorer les performances (38 s → objectif < 24 s).	Équipe	Haute
3	Explorer et benchmarker de nouveaux LLM (Minimax, GPT-5, QwenCoder).	Équipe	Moyenne
4	Intégrer OpenTelemetry pour le monitoring et l'observabilité.	Équipe	Moyenne
5	Mettre en place le <i>reverse proxy</i> (Ollama → GitHub Copilot).	Équipe	Moyenne
6	Trouver un serveur de déploiement pour le projet.	Équipe	Moyenne
7	Intégrer <i>llm-inspector</i> pour la détection de biais.	Équipe	Normale

TABLE 3 – Tableau des actions

6. Processus de développement et collaboration

6.1. Intégration Continue / Déploiement Continu (CI/CD)

Mise en place d'un pipeline **CI/CD via GitHub** pour un développement piloté par les tests (*Test-Driven Development*). Chaque **push** devra déclencher l'exécution automatique de la suite de tests, avec affichage de *badges* de statut dans le dépôt.

6.2. Polyvalence de l'équipe

❗ Il est fortement recommandé que chaque membre de l'équipe soit capable d'implémenter des fonctionnalités dans **chaque composante** du projet, afin de garantir la continuité et la résilience du développement.

7. Rappel de la feuille de route du projet

Le projet s’étend sur **3 mois** (du 6 février au 29 mai 2026), avec une équipe de 3 élèves. Le travail est réparti autour de deux axes principaux :

- **Axe 1** : Implémentation de l’architecture adéquate pour le système.
- **Axe 2** : Phase de test & implémentation des métriques d’évaluation.

Période	Objectifs
Semaine 1	Analyse détaillée du sujet ; prise de contact avec l’encadrant.
Semaine 2	Setup Git ; planning prévisionnel ; sélection de LLM de référence ; premier cas d’usage (tickets IT).
Semaine 3	Revue de la littérature ; construction du dataset (tickets IT fictifs et publics) ; définition des métriques d’évaluation.
Semaines 4 à 8	Mise à jour du planning ; finalisation du dataset ; implémentation des métriques d’évaluation automatiques.
Semaines 9 à 11	Tests des fonctionnalités ; synthèse des résultats ; rédaction du rapport final ; préparation de la soutenance.

TABLE 4 – Planning prévisionnel du projet

 **Avancement actuel** : L’équipe se situe en **semaine 2** du planning prévisionnel. Les démonstrations techniques présentées lors de cette réunion s’inscrivent dans les objectifs de cette phase.

Prochaine réunion

Vendredi 20 février 2026 — 14 h 30 — Microsoft Teams